

Frontend Application

> Relevant source files

This document covers the Vue.js frontend application structure, components, and user interfaces for the MKTY Smart Healthcare System. The frontend serves as the primary user interaction layer, providing web-based interfaces for medical consultations, record management, community features, and administrative functions.

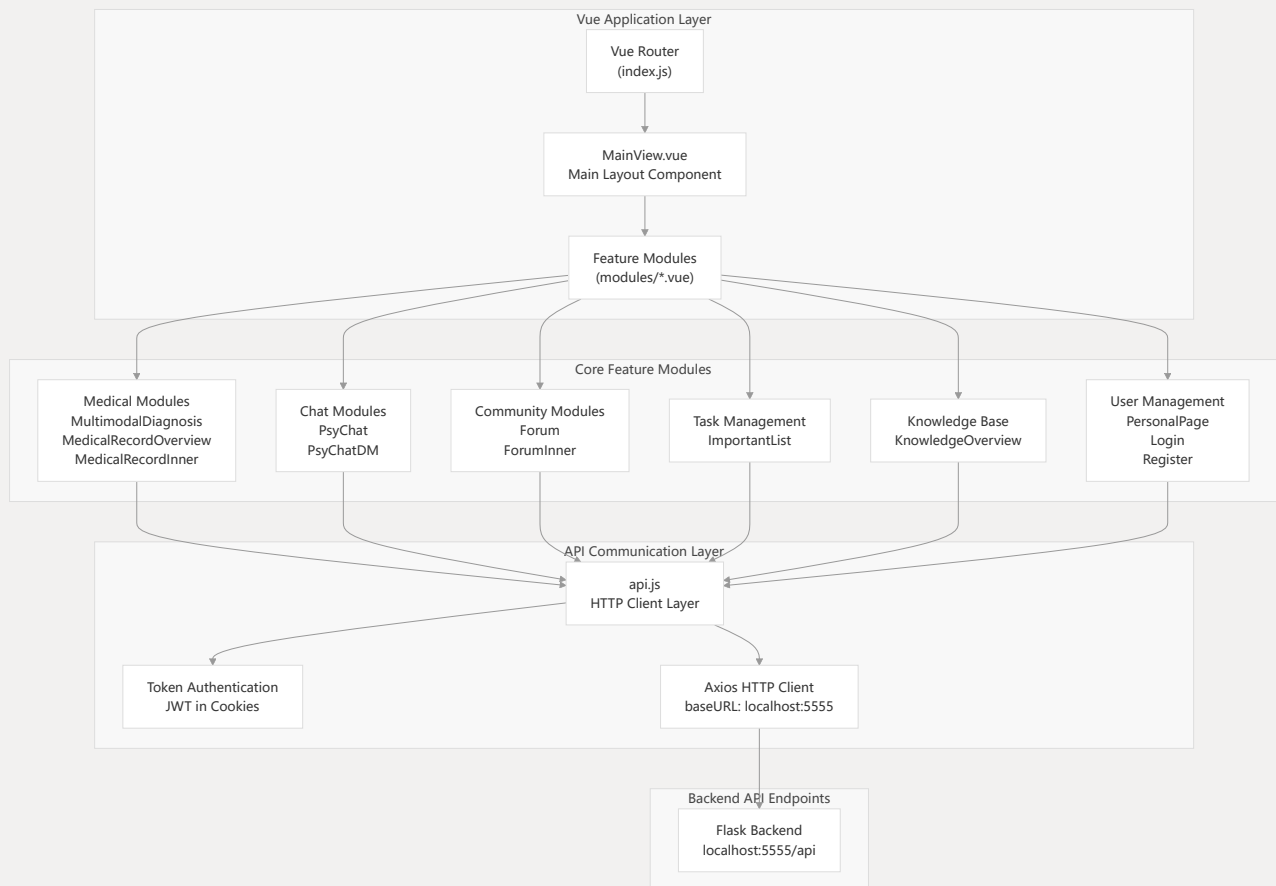
For backend API implementation details, see [Core API Endpoints](#). For AI service integration, see [AI & Machine Learning Services](#). For administrative interfaces, see [Admin Frontend Interface](#).

Technology Stack and Architecture

The MKTY frontend application is built using Vue 3 with a component-based architecture. The technology stack includes:

Technology	Purpose	Version/Library
Vue 3	Core framework	Composition API
Vue Router	Client-side routing	<code>createWebHistory</code>
Axios	HTTP client	REST API communication
Element Plus	UI component library	Form controls, dialogs
Vite	Build tool	Development server
js-cookie	Cookie management	Authentication tokens

Frontend Application Architecture

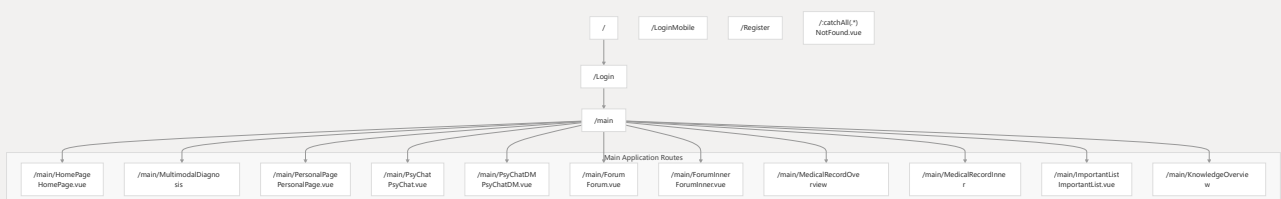


Sources: `frontend/src/router/index.js` 1-158 `frontend/src/api/api.js` 1-542

Routing Configuration and Navigation

The application uses Vue Router with history mode for client-side navigation. The main route structure divides functionality into authenticated and public routes.

Route Structure Mapping



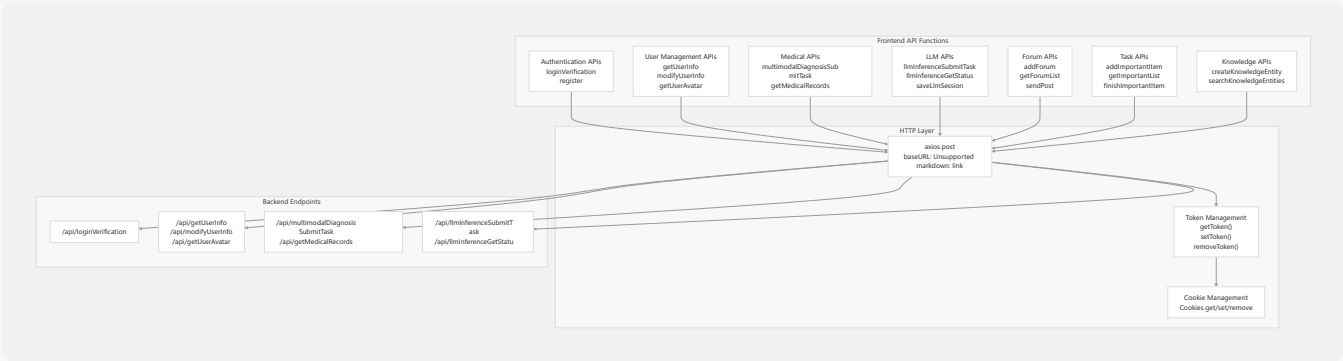
The router configuration includes metadata for dynamic page titles and navigation guards for authentication validation.

Sources: `frontend/src/router/index.js` 5-157

API Communication Layer

The frontend communicates with the backend through a centralized API layer that manages HTTP requests, authentication, and error handling.

API Client Architecture



Authentication Flow

The authentication system uses JWT tokens stored in browser cookies. The `getToken()` function automatically handles token retrieval and validation:

Function	Purpose	Implementation
<code>setToken(token)</code>	Store JWT token	<code>frontend/src/api/api.js</code> 44-54
<code>getToken()</code>	Retrieve and validate token	<code>frontend/src/api/api.js</code> 66-78
<code>removeToken()</code>	Clear authentication	<code>frontend/src/api/api.js</code> 61-63

Sources: `frontend/src/api/api.js` 7-542 `frontend/src/api/api.js` 44-78

Core Component Modules

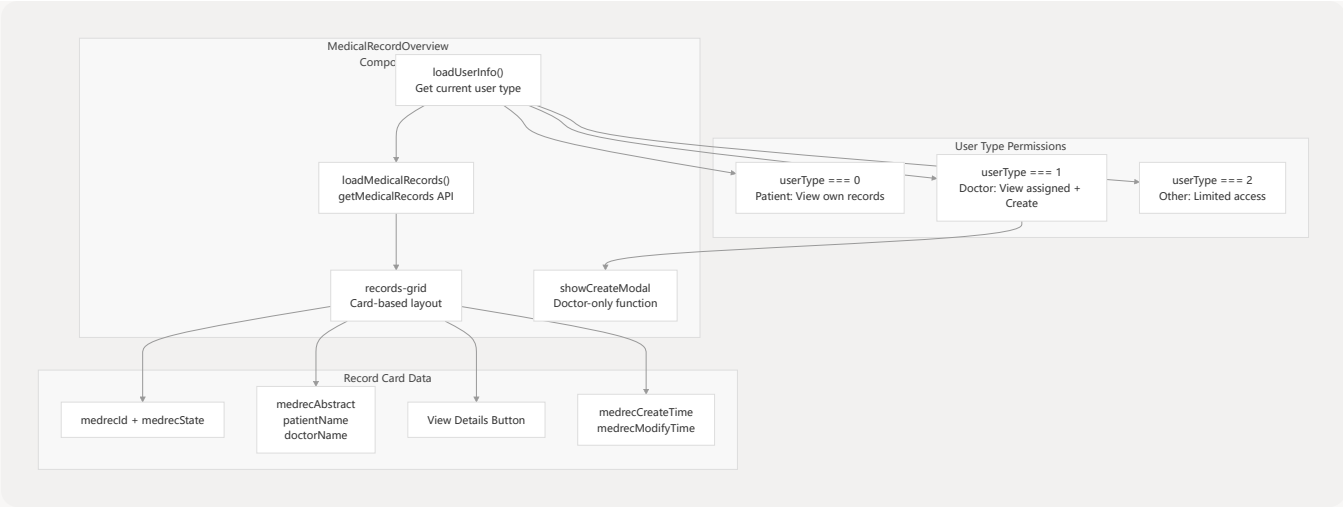
The application is organized into feature-specific Vue components that handle distinct aspects of the healthcare system.

Medical Records Management Components

The medical records system consists of two primary components that handle CRUD operations for patient medical records.

MedicalRecordOverview Component

The `MedicalRecordOverview.vue` component provides a grid-based interface for viewing medical record summaries:



MedicalRecordInner Component

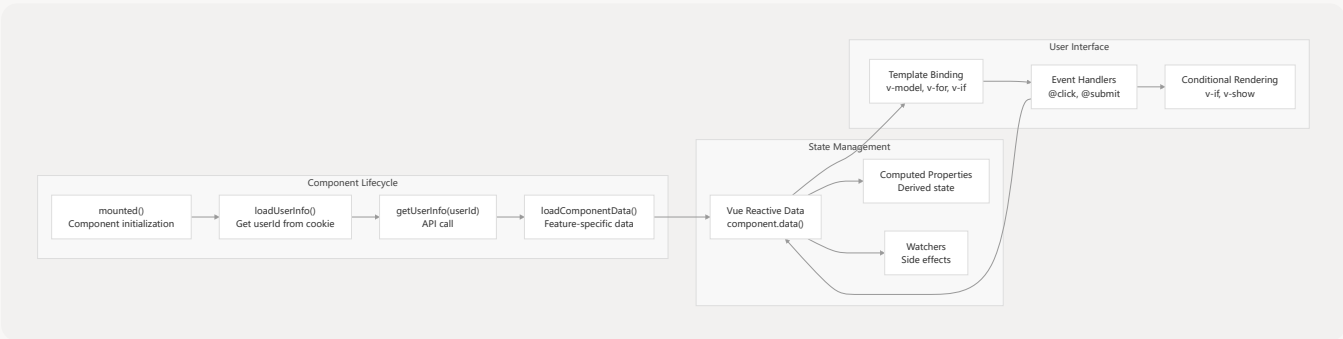
The `MedicalRecordInner.vue` component handles detailed record viewing and editing:

Feature	Implementation	API Calls
Record Loading	<code>loadMedicalRecord()</code>	<code>getMedicalRecord(medrecId)</code>
Content Rendering	<code>marked.parse()</code> + <code>DOMPurify.sanitize()</code>	Markdown to HTML
Edit Permission	<code>canEdit = userType === 1</code>	Doctor-only editing
AI Analysis	<code>mri_aiAncillaryAnalysisDiagnosis()</code>	<code>llmInferenceSubmitTask()</code>
Drug Recommendations	<code>mri_aiAncillaryAnalysisRecommendations()</code>	<code>llmInferenceSubmitTask()</code>

Sources: `frontend/src/components/modules/MedicalRecordOverview.vue` 1-733
`frontend/src/components/modules/MedicalRecordInner.vue` 1-880

Component Data Flow and State Management

The frontend uses Vue's reactive data system for state management within components, with no centralized state store. Data flow follows this pattern:



API Integration Patterns

Components follow consistent patterns for API integration:

- 1. **Authentication Header Injection:** All authenticated requests use `getToken()` for authorization headers
- 2. **Error Handling:** Standardized error handling through `errHandle()` and `successHandle()` utility functions
- 3. **Loading States:** Components maintain loading state during async operations
- 4. **Data Transformation:** API responses are processed and stored in component reactive data

Example API integration pattern from medical records:

```
// Pattern used across components
async loadMedicalRecords() {
  try {
    const res = await getMedicalRecords()
    if (res.data.code === 0) {
      this.medicalRecords = res.data.medicalRecords
    } else {
      errHandle('获取病历列表失败: ' + res.data.msg)
    }
  } catch (error) {
    errHandle('加载病历列表时出错: ' + error.message)
  } finally {
    this.loading = false
  }
}
```

Sources: `frontend/src/components/modules/MedicalRecordOverview.vue` 198-211
`frontend/src/components/modules/MedicalRecordInner.vue` 238-261

Component Communication and Navigation

The frontend uses Vue Router for navigation between components and URL-based parameter passing for component communication.

Inter-Component Navigation Patterns

Navigation Type	Implementation	Example
Programmatic Navigation	<code>this.\$router.push()</code>	Record detail navigation
Query Parameters	<code>this.\$route.query</code>	Passing <code>medrecId</code>
Route Guards	<code>router.beforeEach()</code>	Authentication checks

Navigation Type	Implementation	Example
Dynamic Titles	<code>document.title</code>	Page title updates

The medical records workflow demonstrates typical navigation patterns:

- Overview component navigates to detail: `this.$router.push('/main/MedicalRecordInner?medrecId=${medrecId}')`